Variable Roles in Code Models: Surface Transfer Tracks Language Lineage, Probe Transfer Does Not

Anonymous Author(s)
Affiliation
Address
email

Abstract

If language is a design axis rather than a passive carrier, then changing the language should change what a model can do. We test this in code, where one problem can be written in several languages whose relatedness is fixed independently of our results. Using XLCoST solutions aligned problem by problem across Python, JavaScript and PHP, we fit a variable-role classifier on one language and evaluate on another. A model-free character n-gram baseline with the variable's own name masked is strongly constrained by language relatedness: macro-F1 0.952 between JavaScript and PHP, falling to 0.785 on any transfer to or from Python. A linear probe on a code model's residual stream moves 0.006 across the same boundary. The probe is not reading the input: an untrained network of the same architecture reaches 0.575, cleared in all 18 cells. The base model the code model is continued from is almost as invariant (-0.011), so code-*specialised* pretraining is not the cause. The surface fails not because its discriminative n-grams are missing (70% of weighted mass survives, $r = +0.09$ with transfer) but because they point the other way: the share whose sign flips predicts transfer at $r = -0.98$. The medium governs surface generalisation and leaves the learned role representation largely untouched, a dissociation a behavioural evaluation would report as one degraded number.

1 Introduction

Treating language as a design axis presumes the medium is not neutral: rephrasing a problem changes what a model does with it. Programming languages make that testable in a way natural languages rarely do. Code models demonstrably encode a great deal about programs [Karmakar and Robbes, 2021, Troshin and Chirkova, 2022], but what survives a change of language is a separate question, and the same algorithm can be written in Python, JavaScript and PHP with the semantics fixed, the surface varying along the C-family/off-side-rule split, and the corpora aligned problem by problem. If the medium constrains what a model represents, code is where that can be measured instead of argued.

In natural language, transfer tracks relatedness closely enough that typological distance is used to choose source languages [Lin et al., 2019, Littell et al., 2017], and word-order divergence alone degrades it [Ahmad et al., 2019]. Whether the same holds for programming languages is untested, and a single cross-lingual accuracy conflates the two things that would answer it: how far a surface regularity carries, and how far a learned representation does. Variable roles make the separation tractable. A *role* [Sajaniemi, 2002], such as accumulator, loop iterator or index key, is a property of what a variable does, not of its name or its language, so the medium ought in principle to be irrelevant to it. Whether it is irrelevant *to a model* then has a measurable answer.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

We contribute a controlled measurement of that separation and a mechanism for the part that fails. Over all six ordered pairs of Python, JavaScript, and PHP, a model-free surface baseline tracks language relatedness closely while a linear probe on the residual stream does not. An untrained network of identical architecture separates the trained network's contribution from our inputs', and the base model the code model is continued from shows the effect does not require code specialisation. The surface fails for a reason that is not the obvious one: its features are present in the target and carry the opposite sign.

2 Setup

Corpus and alignment. XLCoST [Zhu et al., 2022] provides parallel solutions to competitive-programming problems in several languages, keyed by a problem identifier that is stable across languages. Every comparison below is *matched*: a classifier trained on A and evaluated on B sees only problems appearing in both, so a drop in transfer cannot be a change of subject matter. Matching is also what restricts the study to three languages. We computed the full pairwise overlap (Appendix A): Python/JavaScript/PHP share $2{,}953$, $1{,}529$, and $1{,}145$ problems, and *no* other pair in XLCoST reaches 500, including same-family pairs such as C# and Java, which share 175. The three-language limit is a property of the benchmark, not a choice.

Roles and occurrences. We extract per-occurrence labels for accumulator, iterator, and index_key from language-specific parsers, cap each role at $3{,}000$ occurrences sampled as whole problems, and use a frozen hash-based split that assigns a problem to the same fold in every language. Test folds hold $1{,}377$–$3{,}341$ occurrences with smallest-class counts of 138–$1{,}645$, so one occurrence of the smallest class changing its prediction moves the surface baseline's macro-F1 by $\rho \in [0.0003, 0.0020]$ and the probe's by $[0.0016, 0.0132]$. Every transfer figure in Table 1 clears its own ρ by more than an order of magnitude. The per-class ablations of Section 5 do not, and we read them only as an upper bound.

What is compared. The *surface baseline* is a character n-gram classifier over the context around an occurrence, with every instance of the variable's own name replaced by a placeholder, fitted on the source language and applied to the target. It involves no model. Its score is the strongest of three masked-context variants per cell, and fixing any single variant in advance gives a *larger* boundary drop (-0.173 to -0.276 against the -0.166 we report), so the reported figure is the conservative choice. The *probe* is a logistic regression on residual-stream activations pooled over the occurrence, with the layer chosen on source-language validation data.

The two are not given the same input. The probe pools over the identifier's own tokens and therefore reads the name. The baseline is forbidden from reading it. We report the identifier alone as a third model-free row of Table 1, so the asymmetry is on the page.

We compare three models: Qwen2.5-Coder-1.5B [Hui et al., 2024]; Qwen2.5-1.5B [Qwen et al., 2024], the general-purpose base model it is continued from, sharing its architecture, scale, and tokenizer but not its code-specialisation stage; and an untrained network with that architecture and randomly initialised weights.

XLCoST distributes Python through a formatted mirror and the other two through a detokenized one, so the lineage boundary is also a formatting boundary. The character analyser discards whitespace before extracting n-grams, and equalising it moves transfer by 0.0000 (Appendix C).

3 The medium is encoded, and organised by lineage

First, does the model represent the medium at all? Applying PCA to last-token residual activations over seven XLCoST languages, language identity is linearly recoverable from the embedding layer onward: PC1 correlates with a static/dynamic typing label at $r = +0.883$ at layer 0, peaks at $|r| = 0.941$ at layer 4 carrying 37% of the variance, and decays to 0.576 by layer 28 (Appendix D).

The typing axis is not privileged. A control rules out reading that as internalised type discipline. Relabelling the same activations under every alternative 4-versus-3 partition of the seven languages

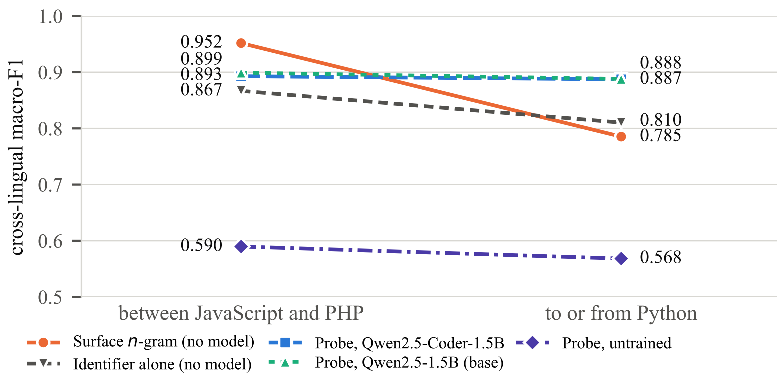

Figure 1: Cross-lingual role transfer, macro-F1 over three roles. Left: the two directions between JavaScript and PHP. Right: the four transfers where Python is one of the two languages. The model-free measures track the boundary; the trained probes do not. Per-cell values in Table 2.

($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls), the true static/dynamic split reaches best-layer F1 1.000 and the arbitrary partitions average 0.996, with 5 of 34 matching or exceeding it. Any grouping is near-perfectly decodable because what the model encodes is language *identity*, and a partition is recoverable as a union of identities whether or not it means anything.

What survives is the arrangement: the two pairs sitting closest are the two that are historically closest, C beside C++ and Java beside C#. The medium is represented sharply and, at least locally, by descent.

4 Role transfer across the lineage boundary

Figure 1 splits the six ordered pairs by whether Python, the one language in the matched triangle outside the C family, is one of the two.

The surface is constrained by the medium. Between JavaScript and PHP the n-gram baseline reaches 0.952 without a model and without seeing the variable's name; transferring to or from Python it loses 0.166. The extreme cell is iterator from PHP to Python, falling to 0.576 against 0.965 from PHP to JavaScript: same source language, same feature family, refit on each pair's own matched set (208 problems against 248). The probe on those cells barely moves (0.865 and 0.851), so the collapse is not an artefact of the smaller set.

The representation is not. The same split costs the code model's probe 0.006, and across all 18 cells its scores are far less dispersed than the masked baseline's (SD 0.028 against 0.100). The probe does not uniformly beat the baseline: it loses all six JavaScript–PHP cells, where n-grams have already saturated. The model contributes nothing where surface similarity is high and a great deal where it is low.

Two controls decide what this means. A flat curve is worthless if the probe reads the input rather than the model, and worthless again if it is flat only because it is uniformly poor. An untrained network of the same architecture bounds both [Hewitt and Liang, 2019, Belinkov, 2022]. The untrained network settles both. It reaches 0.575 against 0.889 and 0.892 for the trained models, gaps of 0.314 and 0.316, positive in *all eighteen* cells for both with a minimum per-cell lift of 0.126. The probes read something the networks learned. It is also flat in group mean, dropping 0.021, though its per-cell scores span 0.417 to 0.780: what is flat is the average, not the network. That is why flatness alone proves nothing. What distinguishes the trained models is being flat *and* high. The untrained network is not at chance either. It clears its own shuffled-label control by 0.111, as random projections of token identity should, so 0.575 rather than 0.5 is the floor a representational claim must clear.

Code specialisation is not the cause. The base model of identical architecture, scale, and tokenizer is nearly as invariant: it moves -0.011 against the code model's -0.006, both an order of magnitude below the surface baseline's -0.166. Whatever makes the role representation insensitive to the medium is not the code-specialisation corpus. No genuinely code-free model exists in this family, so the contrast isolates that stage, not code exposure.

5 Why the surface fails: form–meaning realignment, not missing forms

Calling the drop typological names it without explaining it. Because the baseline fits its vectoriser on the source and only transforms the target, an n-gram the target never produces contributes zero, which makes the mechanism measurable.

The obvious explanation is wrong. Weighting each feature by $|\beta_j|$ times its mean tf-idf, roughly 70% of the source classifier's discriminative mass is realised in *every* target, including the worst (0.697 against 0.735). Surviving mass shows no detectable relationship with transfer ($r = +0.09$, 95% CI $[-0.78, +0.84]$): it stays inside a 0.70–0.86 band while F1 spans 0.58–0.97. The cues are present.

What predicts transfer is whether they mean the same thing (Figure 2). The failure has the shape of a false-friends effect in the classifier's feature space: the forms survive into the target but their mapping to roles reverses, and it is the reversed share rather than the missing share that predicts transfer. Fitting a second classifier on the target labels in the same feature space, the share of target-realised mass whose sign flips between the two tracks transfer strongly ($r = -0.98$, CI $[-1.00, -0.79]$; -0.91 under a source-weighted variant): 5–7% between JavaScript and PHP against 18–37% across the Python boundary, with no overlap between the groups.

The realignment is distributed rather than localised, which is not consistent with the reading "typological" invites. For the iterator role and the two PHP-source pairs that bracket the entire 18-cell spread, masking an orthographic character class on both sides (terminators, braces, brackets, parentheses, or operators) moves transfer by at most 0.021 against a 0.39 gap (Table 5), and none is large relative to its own resolution. Indentation and keywords are not ablated, so this bounds punctuation, not syntax in general. We report the aggregate only: single coefficients of a regularised model over correlated character n-grams are not interpretable one at a time.

Related work. Probing shows code models encode identifier, type and structural information recoverable by linear classifiers [Karmakar and Robbes, 2021, Troshin and Chirkova, 2022]. We ask what survives a change of language, pairing every probe with a model-free baseline on the same occurrences. Multilingual encoders carry a separable language-identity component and a structured geometry [Libovický et al., 2020, Chang et al., 2022, Pires et al., 2019], and multilingual training transfers across programming languages [Ahmed and Devanbu, 2022]. We find that geometry in a code model and show role transfer is largely insensitive to it. We follow the argument that a probe score needs a bound on what architecture and labels alone supply [Hewitt and Liang, 2019, Zhang and Bowman, 2018, Voita and Titov, 2020, Belinkov, 2022].

6 Discussion

Read against the design-axis premise, our results split it in two. The medium constrains surface generalisation, and the model encodes it sharply enough to place historically adjacent languages together (Section 3). Yet it leaves the learned role representation nearly untouched, in a network without code-specialised pretraining. A behavioural evaluation reporting one cross-lingual accuracy would average these into a single degraded number and attribute it to the language, when the language is doing all of its work at the surface.

Limitations. The matched triangle instantiates "closely related" by one pair and the boundary by one language. The axis has two points and no gradient, and Appendix A shows XLCoST admits no wider version. Both trained models are 1.5B parameters from one family with no code-free member, so nothing here speaks to scale.

Role labels come from an AST for Python and regular expressions for the other two, which carve *iterator* differently, so those cells confound the representational effect with a labelling difference. Accumulator and index_key, whose extractors agree, show the same direction (Appendix F). Section 5 rests on one role, two of six pairs and six points, and the probe is correlational.

References

Wasi Uddin Ahmad, Zhisong Zhang, Xuezhe Ma, Eduard Hovy, Kai-Wei Chang, and Nanyun Peng. On difficulties of cross-lingual transfer with order differences: A case study on dependency parsing. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2440–2452, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1253.

Toufique Ahmed and Premkumar T. Devanbu. Multilingual training for software engineering. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1443–1455, Pittsburgh, Pennsylvania, 2022. ACM. doi: 10.1145/3510003.3510049.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

Tyler A. Chang, Zhuowen Tu, and Benjamin K. Bergen. The geometry of multilingual language model representations. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 119–136, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.9.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. doi: 10.48550/arXiv.2409.12186.

Anjan Karmakar and Romain Robbes. What do pre-trained code models know about code? In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1332–1336, Melbourne, Australia, 2021. IEEE. doi: 10.1109/ASE51524.2021.9678927.

Jindřich Libovický, Rudolf Rosa, and Alexander Fraser. On the language neutrality of pre-trained multilingual representations. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1663–1674, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.150.

Yu-Hsiang Lin, Chian-Yu Chen, Jean Lee, Zirui Li, Yuyan Zhang, Mengzhou Xia, Shruti Rijhwani, Junxian He, Zhisong Zhang, Xuezhe Ma, Antonios Anastasopoulos, Patrick Littell, and Graham Neubig. Choosing transfer languages for cross-lingual learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3125–3135, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1301.

Patrick Littell, David R. Mortensen, Ke Lin, Katherine Kairis, Carlisle Turner, and Lori Levin. Uriel and lang2vec: Representing languages as typological, geographical, and phylogenetic vectors. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, 2017. doi: 10.18653/v1/e17-2002.

Telmo Pires, Eva Schlinger, and Dan Garrette. How multilingual is multilingual BERT? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4996–5001, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1493.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Jorma Sajaniemi. An empirical analysis of roles of variables in novice-level procedural programs. In *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments*, pages 37–39, Arlington, VA, USA, 2002. IEEE Computer Society. doi: 10.1109/HCC.2002.1046340.

Sergey Troshin and Nadezhda Chirkova. Probing pretrained models of source codes. In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 371–383, Abu Dhabi, United Arab Emirates (Hybrid), December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.blackboxnlp-1.31. URL `https://aclanthology.org/2022.blackboxnlp-1.31/`.

Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.14.

Kelly W. Zhang and Samuel R. Bowman. Language modeling teaches you more than translation does: Lessons learned through auxiliary syntactic task analysis. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 359–361, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5448.

Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. XLCoST: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474*, 2022. doi: 10.48550/arXiv.2206.08474.

A Pairwise problem overlap in XLCoST

Table 1: Shared problem identifiers between every pair of XLCoST languages, train split. Matched cross-lingual transfer needs a pair to share enough problems to hold out a test fold; only the three bold cells clear 500. No pair outside Python/JavaScript/PHP does, including same-family pairs.

	C++	C#	Java	JavaScript	PHP	Python
C++	—	115	122	75	17	74
C#	115	—	175	75	14	62
Java	122	175	—	85	11	66
JavaScript	75	75	85	—	**1,529**	**2,953**
PHP	17	14	11	**1,529**	—	**1,145**
Python	74	62	66	**2,953**	**1,145**	—

B Full transfer table

C Transfer mechanism

D Language geometry

The typing axis is not privileged. Probing the same activations under every alternative 4-versus-3 partition of the seven languages ($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls) gives best-layer macro-F1 0.9959 on average (min 0.9918, max 1.0000), against 1.0000 for the true static/dynamic split; 5 of the 34 controls match or exceed it. Any grouping of these languages is near-perfectly decodable, so the decodability of the typing split is evidence that language identity is encoded, not that type discipline is. At layer 4, where $|r|$ peaks at 0.941, the remaining components carry almost none of the label: PC2 $r = -0.014$, PC3 $r = +0.026$, PC4 $r = +0.171$.

Table 2: All eighteen transfer cells for Qwen2.5-Coder-1.5B. *Surface* is the strongest masked-context n-gram feature; *name* uses the variable's name alone; *probe* is the residual-stream probe. Majority and shuffled-label controls sit near chance throughout, and ρ is the macro-F1 movement from one test occurrence changing its prediction.

role	pair	n	surface	name	probe	major.	shuf.	ρ
accumulator	JavaScript→PHP	1,976	0.951	0.897	0.904	0.361	0.470	0.0005
accumulator	JavaScript→Python	3,341	0.813	0.873	0.875	0.415	0.473	0.0004
accumulator	PHP→JavaScript	1,903	0.954	0.897	0.918	0.372	0.490	0.0005
accumulator	PHP→Python	1,377	0.782	0.824	0.833	0.276	0.439	0.0008
accumulator	Python→JavaScript	3,294	0.785	0.874	0.918	0.370	0.504	0.0003
accumulator	Python→PHP	1,478	0.736	0.845	0.874	0.271	0.466	0.0007
index_key	JavaScript→PHP	1,976	0.940	0.834	0.897	0.384	0.478	0.0005
index_key	JavaScript→Python	3,341	0.859	0.787	0.883	0.330	0.497	0.0003
index_key	PHP→JavaScript	1,903	0.964	0.838	0.880	0.376	0.516	0.0005
index_key	PHP→Python	1,377	0.839	0.782	0.854	0.387	0.515	0.0008
index_key	Python→JavaScript	3,294	0.871	0.822	0.915	0.314	0.470	0.0003
index_key	Python→PHP	1,478	0.858	0.838	0.874	0.419	0.459	0.0008
iterator	JavaScript→PHP	1,976	0.937	0.862	0.906	0.448	0.488	0.0008
iterator	JavaScript→Python	3,341	0.774	0.733	0.939	0.444	0.472	0.0005
iterator	PHP→JavaScript	1,903	0.965	0.873	0.851	0.446	0.478	0.0008
iterator	PHP→Python	1,377	0.576	0.788	0.865	0.428	0.462	0.0010
iterator	Python→JavaScript	3,294	0.790	0.769	0.918	0.466	0.484	0.0007
iterator	Python→PHP	1,478	0.741	0.788	0.902	0.475	0.438	0.0020

Table 3: Probe transfer per model, averaged within each group. The untrained network shares Qwen2.5-1.5B's architecture and tokenizer with randomly initialised weights; it is the floor a representational claim must clear, and it is itself flat.

model	close pair	Python pairs	all	vs. untrained
Qwen2.5-1.5B	0.899	0.888	0.892	+0.316
Qwen2.5-Coder-1.5B	0.893	0.887	0.889	+0.314
Qwen2.5-1.5B (untrained)	0.590	0.568	0.575	—

E Causal interventions

Scope. Causal interventions were run for the *boolean* role over 5 languages (C++, Java, JavaScript, PHP, Python), 3 models, and 3 intervention modes (ablate, patch, steer), giving 375 layer-wise measurements. They are reported here rather than in the main text because they were run *within* languages and on a role outside the three this paper transfers, so they cannot support its central claim. Specificity is $|\text{clean} - \text{intervened}|$ divided by the same quantity for a random-position control: how much of the effect is specific to the variable's own token positions rather than to editing anything at all. Two columns are given because the layer with the largest effect is generally not the layer with the best specificity, and quoting either alone misleads.

F Role-label extraction

Role labels come from an AST walk for Python and from regular expressions for JavaScript and PHP, and the two paths do not carve *iterator* identically. A C-style counter that also accumulates is dropped as multi-role in JavaScript and PHP but retained as an iterator in Python, so the iterator cells mix the representational effect with a difference in what the extractors call an iterator. The other two roles are less affected and carry the same direction: accumulator transfers at 0.782 php→python against 0.954 php→javascript, and index_key at 0.839 against 0.964. The paper's claim therefore does not rest on the role whose labels agree least across the boundary.

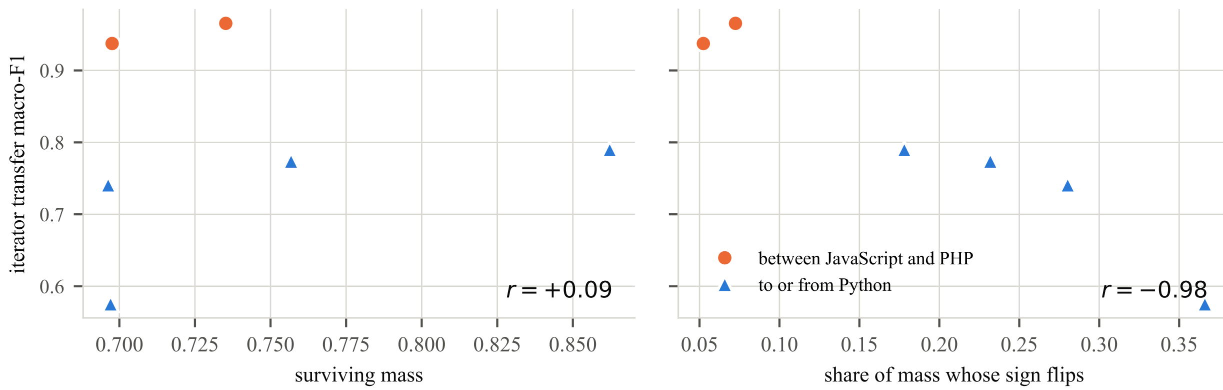

Figure 2: Two candidate explanations for the transfer gap, over the six ordered pairs. Left: how much of the source classifier's discriminative mass the target realises, which is essentially unrelated to transfer. Right: the share of that mass whose sign reverses between a source-fitted and a target-fitted classifier, which tracks it closely. The cues are present in both groups; what changes is where they point.

Table 4: Transfer mechanism for the iterator role. *Surviving* is the share of the source classifier's discriminative mass realised in the target; *flip* is the share of that mass whose sign reverses. Surviving mass is uncorrelated with transfer; flipped mass predicts it almost exactly. Source-weighted variants are given for robustness.

pair	F1	surviving	flip	agree	flip (src-wt.)
PHP→JavaScript	0.965	0.735	0.072	+0.895	0.072
JavaScript→PHP	0.937	0.698	0.052	+0.945	0.068
Python→JavaScript	0.790	0.862	0.178	+0.695	0.211
JavaScript→Python	0.774	0.757	0.232	+0.811	0.241
Python→PHP	0.741	0.696	0.280	+0.528	0.367
PHP→Python	0.576	0.697	0.366	+0.476	0.346

G Reproducibility

Every figure in the main text is regenerated from committed result files by the released pipeline, and a regression test recomputes each of them, including the three correlations of Section 5, from those files rather than quoting them. The appendix tables are emitted by `make_appendix.py` from the same CSVs and are not written by hand.

Models. `Qwen/Qwen2.5-Coder-1.5B` and `Qwen/Qwen2.5-1.5B`, loaded in `float16`. The untrained control instantiates the second model's configuration with randomly initialised weights under a fixed seed, and records its identity as `Qwen/Qwen2.5-1.5B#random-init-s0` so that no downstream artefact can be mistaken for the trained run.

Classifiers. The surface baseline is a `char_wb` tf-idf vectoriser with $n \in [2, 4]$, `min_df` $= 2$ and at most $60{,}000$ features, fitted on the source language only, followed by logistic regression with $C = 4$, balanced class weights, and at most $2{,}000$ iterations. The probe is logistic regression with $C = 1$ on residual activations mean-pooled over the occurrence's token span, with the layer chosen on a source-language validation fold and results averaged over five seeds.

Compute. Activation extraction is the only step needing an accelerator: one forward pass per program, roughly $8{,}700$ programs per model across the three languages, about an hour on a single mid-range GPU, producing stores of roughly 1.6 GB in `float16` at this scale. Everything else, including all surface baselines, the mechanism analyses, and the ablations, is CPU-only and runs in minutes.

Table 5: Masking an entire syntactic character class on both sides of the transfer. No class accounts for more than 0.021 of the 0.39 gap, so the realignment is distributed rather than carried by block delimiters or statement terminators.

pair	masked class	macro-F1	Δ
PHP→JavaScript	terminator	0.9496	-0.0159
PHP→JavaScript	brace	0.9645	-0.0010
PHP→JavaScript	bracket	0.9711	+0.0057
PHP→JavaScript	paren	0.9609	-0.0045
PHP→JavaScript	operator	0.9441	-0.0213
PHP→Python	terminator	0.5786	+0.0030
PHP→Python	brace	0.5750	-0.0006
PHP→Python	bracket	0.5733	-0.0023
PHP→Python	paren	0.5947	+0.0191
PHP→Python	operator	0.5801	+0.0045

Table 6: Point-biserial correlation between PC1 of last-token residual activations and a static/dynamic typing label, by layer, with PC1's explained-variance ratio. The sign of r is arbitrary (PCA fixes the axis only up to reflection, and each layer is refit independently); magnitude is what carries meaning.

layer	$\lvert r \rvert$	EVR	layer	$\lvert r \rvert$	EVR
0	0.883	0.481	15	0.894	0.309
1	0.898	0.447	16	0.882	0.303
2	0.914	0.391	17	0.874	0.296
3	0.939	0.373	18	0.873	0.289
4	0.941	0.370	19	0.873	0.299
5	0.862	0.369	20	0.903	0.291
6	0.828	0.361	21	0.886	0.276
7	0.844	0.342	22	0.859	0.265
8	0.896	0.339	23	0.842	0.252
9	0.919	0.333	24	0.812	0.242
10	0.880	0.327	25	0.774	0.253
11	0.893	0.302	26	0.694	0.255
12	0.917	0.292	27	0.732	0.237
13	0.904	0.314	28	0.576	0.242
14	0.898	0.309			

Data. Section 3 draws on a separate activation-extraction pipeline. Its layer table, principal-component correlations, and bucket-control figures are committed under `results/lp4fm/language_geometry/`, and the values quoted are read from those files. No claim in Sections 4–5 depends on it.

Table 7: Causal interventions on boolean-flag occurrences. *Peak-effect spec.* is specificity at the largest-effect layer; *best spec.* is the maximum over layers, with that layer named.

language	mode	model	peak-effect spec.	best spec. (layer)
C++	ablate	qwen2515b	4.5×	31.9× (L4)
C++	ablate	qwen25coder15b	4.4×	76.2× (L20)
C++	ablate	starcoder27b	4.3×	36.5× (L4)
C++	patch	qwen2515b	3.6×	7.9× (L20)
C++	patch	qwen25coder15b	3.6×	6.9× (L8)
C++	patch	starcoder27b	8.1×	10.4× (L16)
C++	steer	qwen2515b	45.0×	169.8× (L4)
C++	steer	qwen25coder15b	598.5×	598.5× (L24)
C++	steer	starcoder27b	6.9×	341.1× (L4)
Java	ablate	qwen2515b	28.5×	86.0× (L12)
Java	ablate	qwen25coder15b	12.3×	62.4× (L12)
Java	ablate	starcoder27b	27.0×	27.0× (L4)
Java	patch	qwen2515b	6.1×	7.7× (L12)
Java	patch	qwen25coder15b	5.9×	7.7× (L12)
Java	patch	starcoder27b	7.5×	8.6× (L12)
Java	steer	qwen2515b	72.1×	849.0× (L8)
Java	steer	qwen25coder15b	71.9×	186.6× (L0)
Java	steer	starcoder27b	252.0×	252.0× (L24)
JavaScript	ablate	qwen2515b	26.7×	26.7× (L16)
JavaScript	ablate	qwen25coder15b	28.1×	116.3× (L24)
JavaScript	ablate	starcoder27b	27.3×	27.3× (L4)
JavaScript	patch	qwen2515b	6.9×	6.9× (L8)
JavaScript	patch	qwen25coder15b	6.6×	7.4× (L12)
JavaScript	patch	starcoder27b	8.1×	8.2× (L16)
JavaScript	steer	qwen2515b	24.5×	76.4× (L8)
JavaScript	steer	qwen25coder15b	65.2×	149.5× (L4)
JavaScript	steer	starcoder27b	11.2×	60.8× (L28)
PHP	ablate	qwen2515b	15.0×	102.4× (L20)
PHP	ablate	qwen25coder15b	26.5×	79.2× (L20)
PHP	ablate	starcoder27b	19.4×	19.7× (L8)
PHP	patch	qwen2515b	7.2×	7.2× (L4)
PHP	patch	qwen25coder15b	4.7×	7.4× (L0)
PHP	patch	starcoder27b	6.3×	9.3× (L16)
PHP	steer	qwen2515b	190.3×	190.3× (L24)
PHP	steer	qwen25coder15b	11.5×	134.0× (L8)
PHP	steer	starcoder27b	8.7×	100.1× (L4)
Python	ablate	qwen2515b	5.8×	36.7× (L24)
Python	ablate	qwen25coder15b	6.8×	33.1× (L16)
Python	ablate	starcoder27b	26.5×	26.5× (L4)
Python	patch	qwen2515b	6.0×	7.2× (L16)
Python	patch	qwen25coder15b	6.7×	7.8× (L16)
Python	patch	starcoder27b	7.9×	9.7× (L12)
Python	steer	qwen2515b	209.1×	827.9× (L16)
Python	steer	qwen25coder15b	1025.8×	1025.8× (L24)
Python	steer	starcoder27b	8.6×	55.5× (L4)

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the transfer figures, the untrained floor, and the mechanism result, and each is the number reported in Sections 4 and 5. Claims are scoped to three languages, three roles, and two trained models throughout.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: The Limitations paragraph of Section 6 covers the two-point typological axis, the single model family and scale, the extractor-definition difference affecting the iterator role, the scope of the mechanism analysis, and the absence of causal evidence on the cross-lingual cells.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper reports no theoretical results.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

Justification: Section 2 gives the corpus, the matching procedure, the occurrence cap, the split policy, the baseline feature families, and the probe's layer-selection rule. Appendix F names the scripts, the model identifiers, and the classifier settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All result tables are committed as CSVs under `results/lp4fm/`, and the analysis scripts are released. Appendix F gives the commands. An anonymised mirror is provided for the review period.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 2 specifies the cap, the frozen hash split, and the source-validation layer selection. Appendix F gives the n-gram vectoriser configuration and the logistic-regression settings.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: We report resolution (ρ, the macro-F1 movement from one test occurrence changing its prediction) for every cell rather than confidence intervals, and state in Section 2 which comparisons clear it and which do not. Per-cell cluster-bootstrap intervals are computed by the released code but are not reported in this version; the mechanism correlations rest on six ordered pairs and are reported with intervals in Section 5.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix F reports the accelerator used and the wall-clock cost of activation extraction, which dominates. The surface baselines and the mechanism analyses are CPU-only.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work analyses publicly released models on a public benchmark of competitive-programming solutions. It involves no human subjects, no personally identifying data, and no deployment.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The work is an analysis of how existing code models represent variable roles. We identify no societal impact specific to it beyond that of the public models and benchmark it studies.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no data or models. It analyses publicly available ones.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: XLCoST is cited and its licence respected. The two Qwen model releases are cited by name and identifier in Section 2 and Appendix F.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

Justification: The released analysis scripts and result files are documented in Appendix F, including how each table is regenerated from the committed CSVs.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper involves no human subjects, so no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [No]

Justification: Large language models are the object of study, not a component of the method. They were used for writing and editing assistance only, which the question exempts from declaration.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.